SQL Window Functions

LAG & LEAD with PARTITION BY and DATEDIFF | Student Revision Handout

1. The Big Idea

LAG and **LEAD** are *value window functions*. They let a row look at another row and bring that value onto the current row, without losing any rows (unlike GROUP BY).

Function	Looks at	Think of it as
LAG(col)	The **previous** row	Looking **back**
LEAD(col)	The **next** row	Looking **ahead**

Analogy: You are standing in a queue. LAG tells you who stood *before* you. LEAD tells you who stands *after* you. If you are first in the queue there is nobody before you, so you get NULL.

2. Syntax

```
LAG ( column, offset, default_value )  OVER ( PARTITION BY ...  ORDER BY ... )
LEAD( column, offset, default_value )  OVER ( PARTITION BY ...  ORDER BY ... )
```

- **column**: the value you want to fetch from the other row.
- **offset** (optional, default 1): how many rows back (LAG) or forward (LEAD).
- **default_value** (optional, default NULL): what to show when there is no such row.
- **PARTITION BY** (optional): splits the data into groups. LAG/LEAD never look across groups.
- **ORDER BY** (**required**): decides what 'previous' and 'next' mean. Without an order these words have no meaning.

3. Practice Table: sales

emp_name	department	sale_date	amount
Arun	Electronics	2024-01-05	500
Arun	Electronics	2024-01-20	700
Bala	Electronics	2024-01-15	300
Bala	Electronics	2024-02-18	900
Chitra	Furniture	2024-01-08	1200
Chitra	Furniture	2024-02-14	800
Divya	Furniture	2024-01-25	1000
Divya	Furniture	2024-03-09	1500

```
CREATE TABLE sales (emp_name VARCHAR(20), department VARCHAR(20),
                    sale_date DATE, amount INT);

INSERT INTO sales VALUES
('Arun','Electronics','2024-01-05',500),   ('Arun','Electronics','2024-01-20',700),
('Bala','Electronics','2024-01-15',300),   ('Bala','Electronics','2024-02-18',900),
('Chitra','Furniture','2024-01-08',1200),  ('Chitra','Furniture','2024-02-14',800),
('Divya','Furniture','2024-01-25',1000),   ('Divya','Furniture','2024-03-09',1500);
```

4. LAG: Fetch the Previous Row

Question: For each sale, show the previous sale amount of the same employee.

```
SELECT emp_name, sale_date, amount,
       LAG(amount) OVER (PARTITION BY emp_name ORDER BY sale_date) AS prev_amount
FROM sales;
```

emp_name	sale_date	amount	prev_amount
Arun	2024-01-05	500	NULL
Arun	2024-01-20	700	500
Bala	2024-01-15	300	NULL
Bala	2024-02-18	900	300
Chitra	2024-01-08	1200	NULL
Chitra	2024-02-14	800	1200
Divya	2024-01-25	1000	NULL
Divya	2024-03-09	1500	1000

Why NULL? Each employee's first sale has no earlier sale, so there is nothing to fetch.

5. LEAD: Fetch the Next Row

Question: For each sale, show the next sale amount of the same employee.

```
SELECT emp_name, sale_date, amount,
       LEAD(amount) OVER (PARTITION BY emp_name ORDER BY sale_date) AS next_amount
FROM sales;
```

emp_name	sale_date	amount	next_amount
Arun	2024-01-05	500	700
Arun	2024-01-20	700	NULL
Bala	2024-01-15	300	900
Bala	2024-02-18	900	NULL
Chitra	2024-01-08	1200	800
Chitra	2024-02-14	800	NULL
Divya	2024-01-25	1000	1500
Divya	2024-03-09	1500	NULL

Why NULL? Each employee's last sale has no later sale. **Remember:** LAG gives NULL on the first row, LEAD gives NULL on the last row of each group.

6. Offset and Default Value

Replace NULL with 0 using the third argument:

```
SELECT emp_name, sale_date, amount,
       LAG(amount, 1, 0) OVER (PARTITION BY emp_name ORDER BY sale_date) AS prev_amount
FROM sales;
```

Look 2 rows back using offset 2 (Electronics, ordered by date: 500, 300, 700, 900):

```
SELECT sale_date, amount,
       LAG(amount, 2) OVER (PARTITION BY department ORDER BY sale_date) AS two_back
FROM sales WHERE department = 'Electronics';
```

sale_date	amount	two_back
2024-01-05	500	NULL
2024-01-15	300	NULL
2024-01-20	700	500
2024-02-18	900	300

Careful with defaults: If you calculate *change = amount - LAG(amount, 1, 0)*, the first row's change becomes the full amount, which is misleading. For comparisons, leaving NULL is often more honest.

7. Why PARTITION BY Matters

PARTITION BY builds a wall between groups. Without it, the first row of one group wrongly picks up the last value of the previous group.

Wrong: no PARTITION BY (all employees mixed in one list)

```
SELECT emp_name, sale_date, amount,
       LAG(amount) OVER (ORDER BY emp_name, sale_date) AS prev_amount
FROM sales;
```

emp_name	sale_date	amount	prev_amount
Arun	2024-01-05	500	NULL
Arun	2024-01-20	700	500
Bala	2024-01-15	300	700 (Arun's value!)
Bala	2024-02-18	900	300
Chitra	2024-01-08	1200	900 (Bala's value!)
Chitra	2024-02-14	800	1200
Divya	2024-01-25	1000	800 (Chitra's value!)
Divya	2024-03-09	1500	1000

Right: with PARTITION BY emp_name (see section 4). Every employee's first row correctly shows NULL.

Rule of thumb: Ask yourself 'previous *within what*?' Previous sale of the same **employee**? Same **department**? Same **customer**? That answer is your PARTITION BY column.

8. DATEDIFF: Number of Days Between Two Dates

DATEDIFF finds the gap between two dates. The syntax **differs by database**, and the argument order is a common mistake.

Database	Syntax (days)	Example result
MySQL	DATEDIFF(end_date, start_date)	DATEDIFF('2024-01-20','2024-01-05') = 15
SQL Server	DATEDIFF(day, start_date, end_date)	DATEDIFF(day,'2024-01-05','2024-01-20') = 15
PostgreSQL	end_date - start_date	DATE '2024-01-20' - DATE '2024-01-05' = 15
Oracle	end_date - start_date	same as PostgreSQL

Watch the order! MySQL: *end first, then start*. SQL Server: *unit, then start, then end*. If you swap the dates you get a negative number.

9. LAG + DATEDIFF: Days Since the Previous Sale

Question: For each sale, how many days passed since the previous sale in the same department?

Step 1: fetch the previous date with LAG.

```
SELECT department, sale_date,
       LAG(sale_date) OVER (PARTITION BY department ORDER BY sale_date) AS prev_sale_date
FROM sales;
```

Step 2: put the LAG inside DATEDIFF.

MySQL

```
SELECT department, sale_date,
       LAG(sale_date) OVER (PARTITION BY department ORDER BY sale_date) AS prev_sale_date,
       DATEDIFF(sale_date,
                LAG(sale_date) OVER (PARTITION BY department ORDER BY sale_date)) AS days_gap
FROM sales;
```

SQL Server

```
SELECT department, sale_date,
       LAG(sale_date) OVER (PARTITION BY department ORDER BY sale_date) AS prev_sale_date,
       DATEDIFF(day,
                LAG(sale_date) OVER (PARTITION BY department ORDER BY sale_date),
                sale_date) AS days_gap
FROM sales;
```

PostgreSQL

```
SELECT department, sale_date,
       LAG(sale_date) OVER (PARTITION BY department ORDER BY sale_date) AS prev_sale_date,
       sale_date -
         LAG(sale_date) OVER (PARTITION BY department ORDER BY sale_date) AS days_gap
FROM sales;
```

Output (same in every database)

department	sale_date	prev_sale_date	days_gap
Electronics	2024-01-05	NULL	NULL
Electronics	2024-01-15	2024-01-05	10
Electronics	2024-01-20	2024-01-15	5
Electronics	2024-02-18	2024-01-20	29
Furniture	2024-01-08	NULL	NULL
Furniture	2024-01-25	2024-01-08	17
Furniture	2024-02-14	2024-01-25	20
Furniture	2024-03-09	2024-02-14	24

Notes: The first sale in each department has no previous date, so the gap is NULL. Feb 14 to Mar 9 is 24 days because 2024 is a leap year (February has 29 days).

10. LEAD + DATEDIFF: Days Until the Next Sale

Same idea, but looking forward. Subtract in the opposite direction (next date minus current date).

MySQL

```
SELECT department, sale_date,
       LEAD(sale_date) OVER (PARTITION BY department ORDER BY sale_date) AS next_sale_date,
       DATEDIFF(LEAD(sale_date) OVER (PARTITION BY department ORDER BY sale_date),
                sale_date) AS days_to_next
FROM sales;
```

For SQL Server use `DATEDIFF(day, sale_date, LEAD(...))`. For PostgreSQL use `LEAD(...) - sale_date`.

Electronics result: 10, 5, 29, NULL. Furniture result: 17, 20, 24, NULL. (LEAD gives NULL on the last row of each group.)

11. Labelling with CASE

Question: Mark each sale as *First Sale*, *Increased* or *Decreased* compared with the employee's previous sale.

```
SELECT emp_name, sale_date, amount,
  CASE
    WHEN LAG(amount) OVER (PARTITION BY emp_name ORDER BY sale_date) IS NULL
         THEN 'First Sale'
    WHEN amount > LAG(amount) OVER (PARTITION BY emp_name ORDER BY sale_date)
         THEN 'Increased'
    ELSE 'Decreased'
  END AS trend
FROM sales;
```

Result: Arun (First Sale, Increased), Bala (First Sale, Increased), Chitra (First Sale, Decreased), Divya (First Sale, Increased).

12. Common Mistakes

- **Forgetting ORDER BY.** LAG and LEAD need it, since 'previous' depends on the order.
- **Forgetting PARTITION BY.** The first row of one group picks up a value from another group.
- **Swapping DATEDIFF arguments.** MySQL and SQL Server expect opposite orders.
- **Using LAG/LEAD in WHERE.** Not allowed. Put the query in a subquery or CTE, then filter outside.
- **Ties in ORDER BY.** If two rows share a date, the order between them is not guaranteed. Add a tie-breaker column.
- **Using a default of 0 blindly.** It can create misleading differences on the first row.

Filtering on a LAG result (subquery pattern)

```
SELECT * FROM (
  SELECT emp_name, sale_date, amount,
         LAG(amount) OVER (PARTITION BY emp_name ORDER BY sale_date) AS prev_amount
  FROM sales
) t
WHERE amount > prev_amount;       -- sales higher than the previous sale
```

Expected rows: Arun 700, Bala 900, Divya 1500.

13. Quick Cheat Sheet

Need	Use
Previous row's value	LAG(col) OVER (PARTITION BY g ORDER BY d)
Next row's value	LEAD(col) OVER (PARTITION BY g ORDER BY d)
Look n rows away	LAG(col, n) / LEAD(col, n)
Replace NULL	LAG(col, 1, 0)
Change from previous	col - LAG(col) OVER (...)
Days since previous	DATEDIFF(date, LAG(date) OVER (...)) [MySQL]
Days to next	DATEDIFF(LEAD(date) OVER (...), date) [MySQL]

14. Practice Questions

Try these on the **sales** table. Answers are on the next page.

1. Show each sale with the previous sale amount of the same employee.

2. Show each sale with the next sale date of the same employee.

3. Show the difference between the current amount and the previous amount per employee.

4. Show the number of days between each sale and the previous sale in the same department.

5. Show each sale with the amount two sales back in the same department (use offset 2).

6. Label each sale as 'Increased', 'Decreased' or 'First Sale' compared with the employee's previous sale.

7. **Challenge:** Show only the sales where the gap since the previous sale in the department is more than 15 days.

Homework: Build a month-over-month growth report on any dataset using LAG.

Answers (MySQL syntax)

Q1

```
SELECT emp_name, sale_date, amount,
       LAG(amount) OVER (PARTITION BY emp_name ORDER BY sale_date) AS prev_amount
FROM sales;
```

Q2

```
SELECT emp_name, sale_date,
       LEAD(sale_date) OVER (PARTITION BY emp_name ORDER BY sale_date) AS next_sale_date
FROM sales;
```

Q3

```
SELECT emp_name, sale_date, amount,
       amount - LAG(amount) OVER (PARTITION BY emp_name ORDER BY sale_date) AS diff
FROM sales;
```

Q4

```
SELECT department, sale_date,
       DATEDIFF(sale_date,
         LAG(sale_date) OVER (PARTITION BY department ORDER BY sale_date)) AS days_gap
FROM sales;
```

Q5

```
SELECT department, sale_date, amount,
       LAG(amount, 2) OVER (PARTITION BY department ORDER BY sale_date) AS two_back
FROM sales;
```

Q6: see section 11.

Q7

```
SELECT * FROM (
  SELECT department, sale_date,
         DATEDIFF(sale_date,
           LAG(sale_date) OVER (PARTITION BY department ORDER BY sale_date)) AS days_gap
  FROM sales
) t
WHERE days_gap > 15;
```

Expected: Electronics 2024-02-18 (29), Furniture 2024-01-25 (17), 2024-02-14 (20), 2024-03-09 (24).